

Weighted Sum, Bias, Nonlinearity: ML Models and the Anatomy of a Neural Network

Every complicated system is learned one layer at a time. You drive a car without knowing how the engine works; you type on a computer without knowing how the operating system schedules your keystrokes. That is *abstraction*: treating a component as a black box, ignoring its internals, and attending only to how to use it. So far in this course the training process has been exactly such a box. We have said that a feedback loop exists and that the model eventually gets good. This lecture opens a different box — the model itself — and the next one opens training.

The essay proceeds in three movements. First, the landscape: the paradigms under which a model can learn, and what separates a neural network from every model that came before it. Second, a field guide to which model to reach for given which kind of data. Third, the anatomy of the simplest neural network there is — neurons, weights, biases, activations — and the one formula that connects them. If you can explain why a neuron trained to detect the crossbar of a 7 fires on a 5 but not on a 6, you understand neural networks.

I. Three Ways to Learn

Three terms get used interchangeably and should not be. *AI* is any technique that mimics human intelligence. *Machine learning* is the subset that learns from data. *Deep learning* is the subset of that which learns from data using a neural network as the architecture. Cutting across all three is a second axis: the *learning paradigm*, the structure of the learning problem itself. There are three, and any of them can be “deep” — deep supervised learning, deep unsupervised learning, deep reinforcement learning.

Supervised learning

Predict a quantity y from data x . The aspects of x that are useful for the prediction are called *features*; the term is loose and will keep reappearing. The quantity y is the *label*: the ground truth, the correct answer. The defining property of supervised learning is that you have the label, and that is why it is the easiest paradigm. A label is a clean signal about what the model should learn — what was right, what was wrong — and the model uses it as a reference at every step.

Two flavors. If y is discrete (which city is this house in?) the problem is *classification*; if y is continuous (what is the house’s price?) it is *regression*. Supervised learning is what you will see most often, because most of the time we have an answer key, and it is the default for projects in this program.

Unsupervised learning

Now drop the label. Instead of images each tagged *cat* or *dog*, you have only images. One honest response is to go and get labels — entire companies were built a decade ago on paying humans to annotate images for computer vision — but there are also things to do with unlabeled data as it stands. Two are worth knowing by name, though we abstract away how they work.

Clustering groups similar items together. Plot the data and three clouds are visible to the eye; the algorithm’s job is to find those clouds without being told they exist. The standard method is k -means. If your project has unstructured data and you suspect there is hidden regularity in it, clustering is the first thing to try — with the caveat that unsupervised learning is hard in general and clustering is no exception.

Dimensionality reduction projects high-dimensional data down to a low-dimensional space while keeping the

main patterns. The name is more frightening than the idea. Watching a video at 144p instead of HD is dimensionality reduction: you have thrown away most of the numbers and can still tell what is happening. This works because almost all data in the wild is high-dimensional but *lives on a low-dimensional manifold* — picture a wavy blanket in three-dimensional space that can be flattened onto a plane. Nothing in nature optimized pixels, or English, for efficiency, so there is redundancy to squeeze out. The standard algorithms are PCA and t-SNE, each learnable in half an hour with some linear algebra. You have already seen the idea in action: the jump from one-hot token vectors (one dimension per vocabulary entry, roughly 50,000) to learned embedding vectors (a few thousand) in the LLM lecture is dimensionality reduction.

One rule of thumb: you cannot visualize anything above three dimensions, and you should stop trying. Geoffrey Hinton’s advice for picturing a fourteen-dimensional space — visualize three dimensions and say “fourteen” to yourself very loudly — is a joke, and it is the whole of the available technique.

Reinforcement learning

The third paradigm is a different framework altogether. An *agent* acts in an *environment*. Each *action* moves it to a new *state*, and along the way it collects *reward*. What it learns is a *policy*: a rule for which action to take in which state so as to collect as much reward as possible. Deep in the theory there are results showing supervised learning as a special case of RL, but for practical purposes treat it as its own thing.

RL is the paradigm of agentic problems: robotics, games (AlphaGo, from the first lecture), and optimization problems such as trading, where the reward is money. For this course the most relevant use is in post-training language models. RLHF — reinforcement learning from human feedback — pushes a model toward being helpful, honest, and harmless. RLVR — reinforcement learning with verifiable reward — rewards a model for math and logic that can be checked mechanically, and is the engine behind the recent gains in reasoning. One structural difference is worth flagging now: the reward function can be anything at all. It does not have to be differentiable, which the standard training loop of the next lecture requires.

Two caveats, both serious. RL training is unstable: the loss wanders, jumps, and often refuses to come down at all, and it is expensive. The recommendation is blunt. Do not attempt RL unless you have trained models before and have intuition for learning rates and the rest. Otherwise it will simply not work.

A question from the audience: what is epsilon-greedy? It is a strategy for the fundamental RL tension between *exploration* and *exploitation*. Move to a new city and you could eat at the first restaurant you like forever; but to actually optimize your experience you should try many restaurants first — and many dishes at each — and only then settle. Epsilon-greedy is one rule for how much exploring to do.

II. What Makes a Neural Network Different

The language models from yesterday are neural networks. Not all language models are (the older, non-neural ones are no longer used), and not all neural networks are language models. Within neural networks there are *architectures*: recurring patterns of structure that work well. The transformer is the dominant one — the architecture behind essentially all recent progress — and the convolutional network is the other you will meet regularly.

For now a neural network is simply a complicated function: a large collection of weights, trained toward some task, like any other ML model. It differs from classical models in three ways.

First, *automatic feature extraction*. In traditional ML the practitioner decides what the model should pay attention to. Predicting house prices, you might supply square footage, zip code, and whether there is a pool; choosing those is *feature engineering*, and it is a human’s job. A neural network is handed raw data — pixels, text — and discovers the relevant features itself, as a by-product of driving the loss down. The embeddings from yesterday are the example: no one told the model that *king* and *queen* differ by gender. It learned that.

Second, *scalability*. Neural networks keep improving as you add data, parameters, and compute, which is why the field has entered the regime it is in.

Third, *expressiveness*. They model complex nonlinear relationships that simpler models cannot. There is

a theorem behind this, the Universal Approximation Theorem, which is out of scope but there if you are curious.

III. A Brief Review

Before the new material, a fast pass over the first lecture. The key insight of ML is to show the computer examples rather than tell it the steps. The *weights* are the model’s brain — adjustable knobs, like coefficients — and the training loop improves them over and over from data. Once training stops, the frozen weights are just a function from input to output; running that function is *inference*. Even ChatGPT generating an essay is a function predicting the next word.

Because inputs and outputs are numbers, any model can be drawn as a curve, and training is drawing the line of best fit. The goal is to fit the underlying pattern and not the noise: connecting every dot is not generalizing, it is overfitting. The defense is to split data into a *training* set, on which the weights are changed, and a *validation* set, on which performance is measured, because unseen data is the only honest estimate of the real world. Watch both losses. When training loss keeps falling while validation loss turns upward, the model has begun to memorize.

Finally, the reason ML took off is the transformer’s ability to scale. The difference between GPT-2 and GPT-3 was not an architectural idea. It was a bigger model, more data, more compute — and, thanks to the double-descent phenomenon from lecture one, scaling further kept helping rather than overfitting.

IV. Choosing a Model by Data Modality

The first question in choosing a model is what kind of data you have — the *modality* — and then the task, and then the trade-offs within. What follows is exposure, not derivation: a mental map of what exists, so you know where to dig when a project demands it.

Text

For most projects in this program, and in the field, the answer is an LLM. This was not true five years ago, when different architectures were tried for different things; today LLMs are general reasoners, good at nearly everything, and natively multimodal — the commercial models take images as well as text, with dedicated components that encode visual input and feed it into the same reasoning engine.

The one other family still in use is BERT. A BERT model is a language model that understands but does not generate: its output is an *embedding*, a semantically meaningful vector, which you then use for something else.¹ Two uses. *Classification*: run ten thousand movie reviews through BERT and classify each as positive or negative, or score it one to five. *Retrieval*: find the piece of text most relevant to a query. Google Search has used BERT since 2019, one of the first commercial deployments of a language model, and retrieval is what powers RAG — retrieval-augmented generation — in which relevant documents from a private corpus are pulled in at query time to improve an LLM’s answer. A recording on BERT is in the Drive under *Advanced Data / ML Lifecycle*; it is no longer a live research area, but it is good to know.

The barrier to trying any of these models is now very low. Search Hugging Face for “BERT” and you will find the base model and hundreds of fine-tuned variants — emotion classifiers, models for classical Chinese. Ask a coding agent to pull one and run it and you will have a working classifier in fifteen minutes:

```
from transformers import pipeline
classifier = pipeline("sentiment-analysis")      # a fine-tuned BERT-family model
classifier("The pacing dragged but the ending landed.")
```

¹Colloquially “LLM” now means a chat-style generative model. Technically BERT models are large language models too; nobody calls them that anymore.

This is the recurring advice of the lecture: the fastest way to build intuition for a model is to use it.

Visual

Images and video have a different architecture per task. For *classification* (cat or dog) the architecture is the convolutional neural network, with ResNet (2015) and VGG the classic instances. Think of the convolution layers as a long preprocessing stage that extracts visual features, followed by an ordinary neural network. For *object detection* — drawing a box around the dog, the bicycle, the truck — the standard is YOLO, convolutional underneath. For *segmentation*, a stronger task that assigns every pixel to an object (this penguin, that penguin, the sky), the current standard is SAM, Meta’s Segment Anything Model, now in its third generation; U-Nets, the older convolutional approach, have fallen out of fashion. For *generation* the field has moved from GANs, popular a few years ago and now essentially dead, to diffusion models, which start from noise and repeatedly denoise toward an image. Diffusion is genuinely hard to understand in full, and it is not autoregressive — there is no step-by-step token loop — which is why a small group of researchers is bullish on diffusion *language* models: they can emit a whole paragraph at once.

Two more acronyms. A VLM, vision-language model, is by now just a modern multimodal LLM; the visual encoder was the bottleneck as recently as 2024 and is no longer. A VLA, vision-language-action model, extends the transformer to produce actions and is the standard for robotics — a niche in this program, since most such work needs a physical lab, though simulation-only projects are possible.

In all these cases, use a pretrained model. Training even a few-billion-parameter LLM from scratch costs tens of thousands of dollars and produces something far worse than what exists; training a CNN from scratch performs terribly. Take a ResNet pretrained on ImageNet, or its equivalent, and *fine-tune* — continue training from the pretrained weights on your task.

Tabular

Tabular data is a spreadsheet: rows are data points, columns are features. A neural network still works here — the vanilla MLP of the next section — and for time series (a stock price over days) sequence-aware architectures such as LSTMs or the classical ARIMA. There is also a small community building LLMs for tables.

But this is where traditional ML earns its keep. Linear regression, decision trees, random forests, and gradient-boosted trees such as XGBoost are still used, and used seriously. A trader at a quant firm may run a great deal of linear regression, because it is interpretable and, when a model starts losing money, interpretability matters. On Kaggle, boosting variants still win tabular competitions outright. Text and images are too high-dimensional and too complex for these models, which is why everything above was a neural network; on tables, do not discount them.

V. Anatomy of a Neural Network

Now the black box. The running example is the one from 3Blue1Brown’s series: an image of a handwritten digit goes in, and the network must say which of the ten digits it is. A classification problem, on a toy task, small enough to see all the way through.

The high-level goal

Watch a well-trained network and the picture is this. Early neurons respond to tiny fragments — short strokes at particular places. Neurons in the next layer combine those fragments into larger parts: a loop at the top, a vertical line down the right. The final layer combines parts into digits: a loop over a line is a 9. The network decomposes the image into *features that grow larger and more complete layer by layer*, and composes the final prediction out of them. Everything that follows is machinery for making that happen.

Input and output

The input layer is the image itself, flattened. Each pixel is a brightness from 0 to 255; read them off row by row — pixel (1,1), (1,2), ..., then (2,1) — and you have the first column of neurons. The output layer has one neuron per class, ten for digits. This is the same shape as the LLM output from yesterday, a score for every entry in the vocabulary. Next-token prediction is classification.

Neurons and activations

Each neuron in between should, after training, represent a *feature* — once again, loosely, some aspect of the input useful for the prediction. Each neuron carries a number, its *activation*, which measures how present that feature is in the current input. If a neuron has learned the horizontal crossbar of a 7, its activation should be high when the crossbar is there and low when it is not.

Weights and biases

Two kinds of tunable parameter connect the neurons. The terminology is slightly confusing: in the course's general usage, “weights” means every tunable number in the model, biases included. Inside a neural network the word narrows.

A *weight* lives on an edge. There is one for every pair of neurons in adjacent layers, and its magnitude is the strength of the connection: how much this neuron cares about that one. In the vanilla network every neuron in one layer connects to every neuron in the next, so the edges are dense — *fully connected*.

A *bias* lives on a neuron, one per neuron in each *hidden layer* (the layers between input and output). Its purpose is less obvious than the weights', and two arguments help. The first is $y = mx + b$: the bias is the $+b$, an additive offset that gives the function more flexibility, and empirically it improves generalization. The second is biological. A neuron in a brain is binary — it fires or it does not — and it fires only when its input crosses a threshold, which is typically hard to reach. If firing requires $mx + b > \text{threshold}$, then moving the threshold to the other side gives $mx > \text{threshold} - b$: the bias is the negative offset that sets how hard the neuron is to turn on. Both stories point the same way, and biases are used in practice because they work.

The activation function

One more component. Wrapped around each neuron's computation is an *activation function*, whose job is to introduce nonlinearity. The classic choice, and the one in the video, is the sigmoid, which squashes any input into the range 0 to 1. It is almost never used anymore. The modern default is ReLU, which despite the name is an **if** statement:

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & x > 0 \\ 0 & x \leq 0. \end{cases}$$

Positive inputs pass through unchanged; negative inputs become zero. Two stays two; negative five becomes zero. Variants such as GELU and Leaky ReLU are what most people actually run.

Why bother? Because without a nonlinearity the entire network collapses. Strip out the activation functions and write the whole computation as one expression: it is a weighted sum of weighted sums, which is just another weighted sum — linear regression with extra steps. The nonlinearity is what lets the network represent functions that a line cannot.

All of these parameters — every weight, every bias — start out random. They are learned by gradient descent, the subject of the next lecture. For today, abstract that away: these are the knobs, and something turns them.

VI. The Forward Pass

Why is the network structured this way? Because the goal is to *communicate information from one layer to the next* — specifically the useful information. The weights dial in how much each neuron should listen to each neuron before it; the bias dials in how hard it is to switch on. Propagate information through enough layers with the right dials and hierarchical features emerge.

In a vanilla network — an MLP, multilayer perceptron — each neuron communicates with every neuron in the previous layer, and which neurons fire in one layer determines which fire in the next, as governed by the weights and biases. In English, the rule is:

The activation of a neuron is the weighted sum, over every neuron in the previous layer, of that neuron’s activation times the connecting weight; plus a bias; wrapped in the activation function.

That is the whole thing, and the intuition is a weighted average. Each neuron is deciding how much to care about each neuron behind it. Take the neuron for the loop at the top of a 9. That loop is composed of five or so short strokes from the previous layer, so the weights from those five neurons should be large. The weights from every other neuron in the previous layer should be near zero: whether the loop is present has nothing to do with what is happening in the bottom-left corner of the image. Training is the process of dialing those weights to those values.

In math

The same statement in the notation you will meet in a textbook:

$$z_i^{(L)} = \sum_j w_{ij}^{(L)} a_j^{(L-1)} + b_i^{(L)}, \quad a_i^{(L)} = \sigma(z_i^{(L)}).$$

Read it slowly. L indexes the layer; i indexes the neuron within that layer. The sum runs over j , the neurons of the previous layer $L - 1$: $a_j^{(L-1)}$ is the j th previous activation, and $w_{ij}^{(L)}$ is the weight on the edge from neuron j in layer $L - 1$ to neuron i in layer L . Add the bias $b_i^{(L)}$ belonging to the current neuron. The result z is sometimes called the *pre-activation*; pass it through the activation function σ — ReLU, sigmoid, whatever you chose — and out comes the activation $a_i^{(L)}$. It looks worse than it is. Ten minutes of staring, or a conversation with an LLM, and it will resolve into the English sentence above.

That formula handles one neuron. A textbook will usually compress the whole layer into matrix algebra,

$$\mathbf{a}^{(L)} = \sigma\left(W^{(L)}\mathbf{a}^{(L-1)} + \mathbf{b}^{(L)}\right),$$

which computes every neuron in the layer at once. It is the same statement, communicated in a more mathematical way.

Conceptual check

Suppose the network is trained on handwritten digits, and some neuron in the third layer has learned the horizontal crossbar of a 7. Pass in a 6. What is that neuron’s activation — high or low? Low: a 6 has no such bar. Now pass in a 5 whose top stroke sits roughly where the 7’s crossbar would. High: the feature is present, even though the digit is different.

The formula explains why. Split the previous layer’s neurons into two groups. Group A: neurons in the region of the image where the crossbar lives. Group B: everyone else. For group A the weights are high — call them 1, since this neuron cares about that region — but on a 6 the activations there are near 0, because nothing is drawn there. The products are 1×0 , summed: zero. For group B the weights are near 0, because the crossbar neuron does not care what happens elsewhere; the activations may be 0 or 1 depending on where the 6’s strokes fall, but $0 \times$ anything is 0. Add the groups and the weighted sum is close to zero. Low activation.

Feed in the 5 and only one thing changes. In group A some of those zeros become ones — there is ink in the relevant region — so the products are 1×1 , and they add up to something substantial. Group B still contributes nothing, because its weights are still zero. High activation. That is the entire mechanism: the weights select a region, the activations report what is in it, and the sum says whether the feature is present. If this did not land on first reading, the 3Blue1Brown video explains it a second way, and the material is easier on the second pass than the first.

VII. Caveats on the Idealized Picture

The picture above — one clean feature per neuron, composing hierarchically — is what we hope a trained network does. Two facts complicate it.

The first is that a model can reach low validation loss with *no interpretable features at all*. Train a digit classifier, open it up, and the neurons may mean nothing you can name. The subfield of adversarial robustness lives on this gap: it is possible to hand a well-performing classifier an image of pure noise and have it announce, at 95% confidence, that it is a 9. Frontier models turn out to be fairly interpretable when studied carefully, but interpretability is not something training guarantees.

The second is *polysemanticity*. Probe a modern transformer and a single neuron often responds to many unrelated things at once — the loop of a 9, but also dogs, but also the letter A. This is one reason interpretability is a research field rather than a lookup. The current technique for untangling it is the sparse autoencoder, which decomposes polysemantic activations into a sparser set of one-feature-per-direction representations. Anthropic’s interpretability publications are the place to see this done on frontier models, and there is a recording from one of the program’s mentors in the Drive.

VIII. From the Q&A

Which activation function should I use? ReLU or GELU. Almost nothing else is used. The reason previews tomorrow’s lecture: training multiplies many numbers together, and if they are small the product vanishes toward zero, if large it explodes — the vanishing and exploding gradient problems. ReLU-family functions are better behaved computationally than the sigmoid, however well-motivated the sigmoid is in theory.

What determines the weights? Nothing you set. They are randomly initialized and then tuned by the training loop to minimize loss. You start the loop; the model learns the values.

Does a neuron’s importance determine its activation? No. Activation is only how present the feature is in the current input. Some neurons do matter more than others, and there is research on identifying and pruning the ones that do not — reaching the same performance with far fewer neurons — but that is a separate question.

What about a short 7 versus a tall 7? The crossbar lands on different pixels, so different input neurons see it. How does the network cope? In a plain MLP, imperfectly. This is precisely the motivation for the convolutional network: by convolving and pooling over neighborhoods of nearby pixels, a CNN learns patterns that survive small translations and perturbations, which is why it beats an MLP on images.

Why not set the weights directly to wherever the loss is minimized? Because you cannot see the loss surface. Finding the maximum of a 100,000-element array by inspection is not possible; you search. The loss landscape has millions of dimensions and no one has a map of it. You try values and follow the slope, which is tomorrow’s topic.

Should I deliberately overfit at first? Yes, that is a common strategy — it appears in Karpathy’s training recipe — and it confirms the pipeline can learn at all before you pull back.

Where to Go From Here

Watch the 3Blue1Brown neural network video if you have not; it covers today's second half a different way. The attention mechanism, the defining feature of the transformer, remains a black box. The recommended lecture series on it will take several hours to truly absorb, and it is optional: benchmarking, prompting, and decoding projects can treat the model as a black box, while interpretability and efficiency projects cannot. Watch it if your project needs it.

The broader advice is the same throughout. Use the models. The barrier to running a BERT classifier, a segmentation model, or a diffusion model is now a natural-language request to a coding agent, and fifteen minutes of trying beats an hour of watching. It is easy to feel you understand something after a lecture and discover otherwise the moment you have to explain it or run it. Do not try to master every architecture horizontally; pick a project and go deep in one place. Research ideas begin with the audition process at the start of week 3 and should be locked in by week 5 or 6.

Step back, and the lecture reduces to one sentence. A neural network is a stack of layers in which every neuron computes a weighted sum of the layer before it, adds a bias, and applies a nonlinearity — and, trained well, that repeated operation turns pixels into strokes, strokes into loops, and loops into a 9. Tomorrow: how the weights get trained.